

# Capítulo 2: Ferramentas Computacionais para Biologia de Sistemas

Bioinformática para Biologia de Sistemas

Ney Lemke

DBF-Unesp

23 de maio de 2026

# Sumário

# Por que um biólogo precisa entender computadores?

## Objetivos de Aprendizagem

- Entender os componentes fundamentais de um computador
- Conhecer as arquiteturas disponíveis hoje para ciência
- Saber escolher a ferramenta certa para cada problema
- Dominar o fluxo de trabalho científico reprodutível

## Importante!

Você não precisa ser programador para fazer ciência computacional — mas precisa entender o **vocabulário** e as **ferramentas** do ambiente digital.

# Os componentes fundamentais

## CPU — Unidade Central de Processamento

- O “cérebro” do computador
- Executa instruções sequencialmente
- 4–64 núcleos em desktops modernos
- Ótima para tarefas complexas e variadas

## RAM — Memória de Acesso Aleatório

- Área de trabalho temporária
- Dados desaparecem ao desligar
- 8–256 GB em estações de trabalho

## Armazenamento (SSD/HDD)

- Dados persistentes
- SSD: rápido (GB/s), caro
- HDD: lento, barato, grande capacidade

## Analogia Biológica

CPU = ribossomo

RAM = citoplasma (área de trabalho)

SSD = DNA (memória permanente)

# GPU: processamento paralelo massivo

## Definição: GPU — Graphics Processing Unit

Processador com milhares de núcleos simples, originalmente criado para gráficos, hoje essencial para Machine Learning, simulações moleculares e análise de imagens biomédicas.

### CPU vs GPU:

|                   | CPU     | GPU          |
|-------------------|---------|--------------|
| Núcleos           | 4–64    | 3 000–16 000 |
| Velocidade/núcleo | Alta    | Baixa        |
| Paralelismo       | Baixo   | Altíssimo    |
| Memória           | GBs RAM | GBs VRAM     |

### Uso em Biologia

- AlphaFold2 (dobramento de proteínas)
- Redes neurais para imagens de microscopia
- Dinâmica molecular (GROMACS, NAMD)
- Sequenciamento por deep learning

# Arquiteturas de computação disponíveis hoje

## Laptop / Desktop

- Uso pessoal, análises pequenas
- Python, R, Jupyter local
- Limitado por RAM e CPU

## Cluster / HPC

- Centenas a milhares de CPUs
- Jobs enviados via fila (SLURM)
- Usado para simulações longas

## Workstation

- 32–512 GB RAM, GPU dedicada
- Análises de tamanho médio
- Compartilhado no laboratório

## Cloud (AWS, GCP, Azure)

- Escala sob demanda
- Paga pelo uso (por hora)
- GPUs disponíveis instantaneamente

# Qual arquitetura usar?

| Tarefa                 | Arquitetura recomendada | Exemplo              |
|------------------------|-------------------------|----------------------|
| Aprender Python/R      | Laptop / Google Colab   | Análise exploratória |
| Análise de RNA-seq     | Workstation ou cluster  | DESeq2, edgeR        |
| Dobramento de proteína | GPU (Colab Pro / HPC)   | AlphaFold2           |
| Genômica populacional  | HPC (cluster)           | GATK, VCF pipeline   |
| Simulação de rede      | Workstation             | NetworkX, igraph     |
| Deploy de modelo ML    | Cloud (AWS/GCP)         | API de predição      |

## Importante!

Comece sempre no ambiente mais simples. Só migre para HPC/cloud quando o problema **realmente** exigir.

# O que é um Sistema Operacional?

## Definição: Sistema Operacional (SO)

Camada de software que gerencia o hardware do computador e fornece serviços para os programas. É o intermediário entre você, seus programas e o processador, a memória e o disco.

### O que o SO faz:

- Gerencia processos (quem usa a CPU)
- Controla memória RAM
- Organiza o sistema de arquivos
- Gerencia dispositivos (teclado, rede, GPU)
- Fornece interface ao usuário (gráfica ou terminal)

### Analogia

O SO é como o sistema nervoso autônomo do computador: você não precisa pensar em como o coração bate (CPU executa instruções) — ele cuida disso enquanto você foca no que importa.



# Windows, Linux e macOS: as três famílias

|                       | Windows    | Linux                       | macOS               |
|-----------------------|------------|-----------------------------|---------------------|
| Criador               | Microsoft  | Comunidade (Linus Torvalds) | Apple               |
| Custo                 | Pago       | Gratuito                    | Incluso no Mac      |
| Código-fonte          | Fechado    | Aberto                      | Parcialmente aberto |
| Uso em servidores     | Raro       | Dominante (90%+)            | Raro                |
| Uso em desktops       | Dominante  | Crescente                   | ~15%                |
| Terminal padrão       | PowerShell | bash/zsh                    | zsh                 |
| Instalação de pacotes | .exe       | apt/yum/conda               | brew/conda          |

## Importante!

Servidores, clusters e nuvem rodam **Linux**. Saber o básico de Linux é indispensável para bioinformática.

# Linux: distribuições mais usadas em ciência

## O que é uma distribuição Linux?

O Linux é apenas o **kernel** (núcleo). Uma distribuição empacota o kernel com ferramentas, gerenciador de pacotes e interface gráfica. Existem centenas — cada uma com foco diferente.

## Windows com Linux: WSL

O **Windows Subsystem for Linux (WSL)** permite rodar Ubuntu dentro do Windows sem dual-boot — boa opção para quem precisa de Windows mas quer usar ferramentas Linux.

## Mais usadas em bioinformática

- **Ubuntu** — mais fácil para iniciantes
- **Debian** — estável, base do Ubuntu
- **CentOS / Rocky Linux** — padrão em clusters HPC
- **Fedora** — atualizado, bom para workstations

## Dica prática

Se você usa Windows, instale o WSL2 + Ubuntu. Se usa Mac, o terminal já é Unix-compatível — você está a um passo do ambiente científico.

# Software Livre vs. Software Proprietário

## Definição: Software Livre (Free/Open Source Software — FOSS)

Software cujo código-fonte é público, podendo ser estudado, modificado e redistribuído por qualquer pessoa, respeitando a licença. “Free” significa **liberdade**, não necessariamente gratuito.

### Software Livre em biologia

- Python, R, Julia — linguagens
- Linux, Ubuntu — sistemas operacionais
- BLAST, BWA, GATK — ferramentas bioinformáticas
- Bioconductor, BioPython — bibliotecas
- Galaxy, Snakemake — plataformas de análise

### Software Proprietário em biologia

- Microsoft Office, Windows
- MATLAB (licença cara)
- CLC Genomics Workbench
- Geneious (análise de sequências)
- GraphPad Prism (estatística)

# Por que Software Livre importa para a ciência?

## Vantagens científicas do FOSS

- **Reprodutibilidade:** qualquer pesquisador pode inspecionar o código
- **Auditabilidade:** algoritmos podem ser verificados e corrigidos
- **Sem vendor lock-in:** você não perde seus dados se a empresa fechar
- **Custo zero:** acessível a pesquisadores de qualquer país
- **Colaboração global:** comunidades como Bioconductor têm milhares de contribuidores

## Tipos de licença comuns

- **MIT / BSD** — use como quiser, inclusive em software proprietário
- **GPL** — derivados também devem ser livres (“copyleft”)
- **Apache 2.0** — permissiva, com proteção de patentes
- **CC BY** — para dados e artigos (Creative Commons)

## Exemplo: Impacto real

O Linux, Python, R e toda a pilha de bioinformática moderna são software livre. A ciência aberta depende de ferramentas abertas.

# O terminal: por que não fugir dele

## Definição: Terminal / Shell

Interface de texto para interagir com o sistema operacional. No Linux/Mac é o `bash` ou `zsh`; no Windows é o PowerShell ou WSL (Windows Subsystem for Linux).

## Por que usar o terminal?

- Automatiza tarefas repetitivas
- Necessário para usar clusters e Docker
- Permite trabalhar em servidores remotos
- Reprodutibilidade: comandos são registráveis

## Analogia

O terminal é como o pipetador automático da computação: a primeira vez parece complicado, mas depois de aprender, você nunca mais quer fazer na mão.

# Comandos essenciais do terminal

```
1 # Ver onde estou
2 pwd
3
4 # Listar arquivos
5 ls -lh
6
7 # Entrar em uma pasta
8 cd minha-analise/
9
10 # Criar pasta
11 mkdir resultados
12
13 # Copiar arquivo
14 cp dados.csv resultados/
15
16 # Mover/renomear
17 mv draft.py analise.py
18
19 # Ver conteudo de arquivo
20 cat sequencias.fasta | head -20
```

Listing 1: Navegação e arquivos

```
1 # Buscar texto em arquivo
2 grep "ATGC" sequencias.fasta
3
4 # Contar linhas
5 wc -l dados.csv
6
7 # Ver processos rodando
8 top
9
10 # Rodar script Python
11 python3 analise.py
12
13 # Ver historico de comandos
14 history | grep python
```

Listing 2: Busca e processos

# Conexão remota com SSH

## Definição: SSH — Secure Shell

Protocolo para acessar computadores remotos (clusters, servidores) com segurança, como se você estivesse na frente do terminal deles.

```
1 # Conectar a um servidor remoto
2 ssh usuario@cluster.unesp.br
3
4 # Copiar arquivos para o servidor
5 scp meu_script.py usuario@cluster.unesp.br:~/analises/
6
7 # Copiar resultados de volta
8 scp usuario@cluster.unesp.br:~/resultados/output.csv .
9
10 # Transferencia de pastas inteiras
11 scp -r pasta_local/ usuario@servidor:~/destino/
```

## Dica prática

Configure chaves SSH (ssh-keygen) para não precisar digitar senha toda vez.

# O que é uma linguagem de programação?

## Definição: Linguagem de Programação

Um conjunto de regras e vocabulário que permite dar instruções precisas a um computador. É uma tradução entre a lógica humana e as operações binárias da máquina.

## Como funciona?

1. Você escreve código em texto legível (ex: Python)
2. Um **interpretador** ou **compilador** traduz para linguagem de máquina
3. O processador executa as instruções

## Exemplo: Analogia

Uma linguagem de programação é como um protocolo de laboratório: você descreve cada passo com precisão e o “assistente” (computador) executa exatamente o que foi escrito — sem imprevisto.



# Python: a linguagem do cientista moderno

## Por que Python?

- Sintaxe próxima ao inglês — fácil de aprender
- Maior ecossistema científico do mundo
- Comunidade enorme com tutoriais em tudo
- Gratuito e open-source
- Integra com R, C++, Java

## Bibliotecas essenciais para biologia:

- numpy, pandas — dados tabulares
- matplotlib, seaborn — visualização
- scipy — estatística e matemática
- biopython — sequências biológicas
- scanpy — single-cell RNA-seq

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Ler dados de expressao genica
5 dados = pd.read_csv("expressao.csv")
6
7 # Ver as primeiras linhas
8 print(dados.head())
9
10 # Histograma de expressao
11 dados["gene_A"].hist(bins=30)
12 plt.xlabel("Expressao (TPM)")
13 plt.ylabel("Frequencia")
14 plt.show()
```

Listing 3: Exemplo simples

# R: o poder da estatística

## Quando usar R?

- Análises estatísticas avançadas
- Bioconductor: maior coleção de ferramentas bioinformáticas
- Publicações que exigem R (DESeq2, edgeR, limma)
- Visualizações publicação-ready com ggplot2

## Pacotes Bioconductor essenciais

- DESeq2 — RNA-seq diferencial
- edgeR — contagens de RNA
- limma — microarray e RNA-seq
- clusterProfiler — enriquecimento
- Seurat — single-cell

```
1 # Instalar pacote
2 install.packages("ggplot2")
3 library(ggplot2)
4
5 # Ler dados
6 dados <- read.csv("expressao.csv")
7
8 # Grafico de dispersao
9 ggplot(dados,
10        aes(x=condicao_A,
11            y=condicao_B)) +
12   geom_point(alpha=0.5) +
13   geom_smooth(method="lm") +
14   theme_minimal()
```

Listing 4: R básico

# Python vs R: quando usar cada um?

| Tarefa                           | Python | R  |
|----------------------------------|--------|----|
| Machine learning / deep learning | ✓✓     | ○  |
| RNA-seq diferencial              | ○      | ✓✓ |
| Visualização para publicação     | ✓      | ✓✓ |
| Automação e pipelines            | ✓✓     | ○  |
| Análise de redes                 | ✓✓     | ✓  |
| Estatística clássica             | ✓      | ✓✓ |
| Single-cell (scRNA-seq)          | ✓✓     | ✓✓ |
| Bioinformática geral             | ✓✓     | ✓✓ |

## Importante!

✓✓ = excelente    ✓ = bom    ○ = possível, mas não ideal

**Na prática: aprenda Python primeiro, adicione R conforme a necessidade.**

# Outras linguagens relevantes

## Julia

- Velocidade de C, sintaxe de Python
- Crescendo em modelagem matemática
- BioJulia para bioinformática
- Ideal para simulações numéricas pesadas

## SQL

- Consultar bancos de dados biológicos
- Usado no backend do UCSC, Ensembl
- `SELECT * FROM genes WHERE organism='Homo sapiens'`

## Bash / Shell Script

- Automatizar pipelines no terminal
- Processar arquivos FASTQ, VCF em lote
- Indispensável para HPC

## C / C++

- Núcleo de ferramentas como BWA, GATK
- Não precisa aprender — mas ajuda a entender

# Jupyter Notebook: ciência interativa

## Definição: Jupyter Notebook

Ambiente de computação interativa que combina código, texto, equações e visualizações em um único documento. O padrão moderno para ciência de dados reproduzível.

### Vantagens:

- Executa código célula por célula
- Visualizações embutidas nos resultados
- Mistura código com documentação
- Exporta para PDF, HTML, slides
- Base do Google Colab

## Instalação

```
pip install jupyterlab  
jupyter lab
```

## Limitação importante

Notebooks fora de ordem de execução podem dar resultados errados. Sempre rode **Kernel** → **Restart & Run All** antes de compartilhar.

# Google Colab: o laboratório na nuvem

## O que é o Google Colab?

Jupyter Notebook hospedado pelo Google, gratuito, sem instalação, com acesso a GPUs e TPUs. Ideal para começar sem configurar nada.

### Versão gratuita:

- GPU T4 por até 12h/sessão
- 12 GB RAM
- 15 GB Google Drive integrado
- Colaboração em tempo real

### Como acessar:

1. Acesse [colab.research.google.com](https://colab.research.google.com)
2. Faça login com conta Google
3. Crie um novo notebook
4. Habilite GPU: Runtime → Change runtime type

## Exemplo: Dica

Para rodar AlphaFold2, ESMFold e ferramentas de deep learning em biologia, o Colab Pro (\$9,99/mês) com GPU A100 é a opção mais acessível para pesquisadores.

# O problema das dependências

## Cena comum em laboratório

“O script funcionava no meu computador, mas no servidor ele dá erro na importação do pandas...”

## Por que isso acontece?

- Diferentes versões de pacotes podem ser incompatíveis
- Projeto A precisa de `numpy 1.24`, projeto B precisa de `numpy 2.0`
- Instalar tudo globalmente causa conflitos

## Definição: Ambiente Virtual

Um diretório isolado com sua própria instalação do Python e pacotes. Cada projeto tem o seu ambiente, evitando conflitos.

# Conda: o gerenciador de ambientes do cientista

```
1 # Criar ambiente com Python 3.12
2 conda create -n meu-projeto \
3     python=3.12
4
5 # Ativar ambiente
6 conda activate meu-projeto
7
8 # Instalar pacotes
9 conda install numpy pandas \
10     matplotlib scipy biopython
11
12 # Listar ambientes
13 conda env list
14
15 # Exportar para reproducao
16 conda env export > environment.yml
17
18 # Recriar em outra maquina
19 conda env create -f environment.yml
```

Listing 5: Criando e usando ambientes

## Por que Conda?

- Gerencia Python e dependências não-Python (C libraries, CUDA)
- Canal bioconda: 8 000+ ferramentas bioinformáticas prontas
- Instalação simples: miniforge (recomendado)

## Instalar BWA via bioconda

```
conda install -c bioconda bwa
```



# pip e venv: a alternativa leve

```
1 # Criar ambiente virtual
2 python3 -m venv .venv
3
4 # Ativar (Linux/Mac)
5 source .venv/bin/activate
6
7 # Ativar (Windows)
8 .venv\Scripts\activate
9
10 # Instalar pacotes
11 pip install numpy pandas scipy
12
13 # Salvar dependências
14 pip freeze > requirements.txt
15
16 # Instalar em outro lugar
17 pip install -r requirements.txt
```

Listing 6: Usando venv + pip

## Quando usar venv + pip?

- Projetos puramente Python
- Ambientes mais leves que conda
- Integração com PyPI (200k+ pacotes)

|                 | conda | venv |
|-----------------|-------|------|
| Leve            | –     | ✓    |
| Bioconda        | ✓     | –    |
| Não-Python libs | ✓     | –    |
| Universalidade  | ✓     | ✓    |

# uv: a nova geração ultrarrápida

```
1 # Instalar o uv (Linux/Mac)
2 curl -LsSf https://astral.sh/uv/
   install.sh | sh
3
4 # Criar ambiente virtual
5 uv venv
6
7 # Ativar (Linux/Mac)
8 source .venv/bin/activate
9
10 # Instalar pacotes (10-100x mais
   rapido)
11 uv pip install numpy pandas scipy
12
13 # Exportar dependencias
14 uv pip freeze > requirements.txt
```

Listing 7: Usando uv

## O que é o uv?

- Escrito em Rust pela Astral
- Substitui drop-in para pip e venv
- **Extremamente rápido** (cache global eficiente)
- Resolve dependências com maior precisão

## Por que mudar?

A comunidade Python está adotando ferramentas em Rust pela sua velocidade brutal. O uv acaba com a espera de minutos na instalação de pacotes grandes em ciência de dados.

# Comparação: conda vs pip/venv vs uv

| Característica         | conda          | pip+venv      | uv                  |
|------------------------|----------------|---------------|---------------------|
| Velocidade             | Lento          | Média         | <b>Ultrarrápido</b> |
| Linguagem base         | Python/C       | Python        | Rust                |
| Leveza                 | Pesado (GBs)   | Leve (MBs)    | <b>Super leve</b>   |
| Bibliotecas Não-Python | ✓ (Bioconda)   | –             | –                   |
| Adoção                 | Bioinformática | Padrão legado | <b>Novo Padrão</b>  |

## Importante!

**Qual escolher?** Use **conda** se sua análise exigir pacotes não-Python complexos (ex: bwa, samtools via Bioconda). Para projetos focados apenas em bibliotecas Python (ex: pandas, scanpy), use **uv** pela sua velocidade imbatível e simplicidade.

# O que é controle de versão?

## Sem controle de versão

- `analise_final.py`
- `analise_final_v2.py`
- `analise_final_corrigida.py`
- `analise_USAR_ESSE.py`
- `analise_USAR_ESSE_2.py`

## Com Git

- Um único arquivo: `analise.py`
- Histórico completo de todas as versões
- Quem mudou o quê e quando
- Voltar a qualquer versão anterior
- Colaboração sem conflito

## Definição: Git

Sistema de controle de versão distribuído, criado por Linus Torvalds (2005). Registra cada mudança em arquivos ao longo do tempo, como um “histórico infinito de Ctrl+Z”.

# Comandos Git essenciais

```
1 # Iniciar repositorio
2 git init
3
4 # Ver estado atual
5 git status
6
7 # Adicionar arquivo ao stage
8 git add analise.py
9
10 # Salvar versao (commit)
11 git commit -m "Adiciona analise de
    expressao"
12
13 # Ver historico
14 git log --oneline
15
16 # Voltar a versao anterior
17 git checkout abc1234
```

Listing 8: Fluxo básico do Git

```
1 # Clonar repositorio existente
2 git clone https://github.com/
3     usuario/projeto.git
4
5 # Enviar para GitHub
6 git push origin main
7
8 # Baixar atualizacoes
9 git pull
10
11 # Criar ramo separado
12 git checkout -b nova-analise
13
14 # Unir ramo ao principal
15 git merge nova-analise
```

Listing 9: Colaboracao com GitHub

# GitHub: muito além do armazenamento de código

## O que o GitHub oferece?

- Hospedagem gratuita de repositórios
- **Issues**: rastreamento de bugs e tarefas
- **Pull Requests**: revisão de código em equipe
- **Actions**: automação de testes e pipelines
- **Pages**: publicação de sites e relatórios
- **Releases**: versões publicadas de software

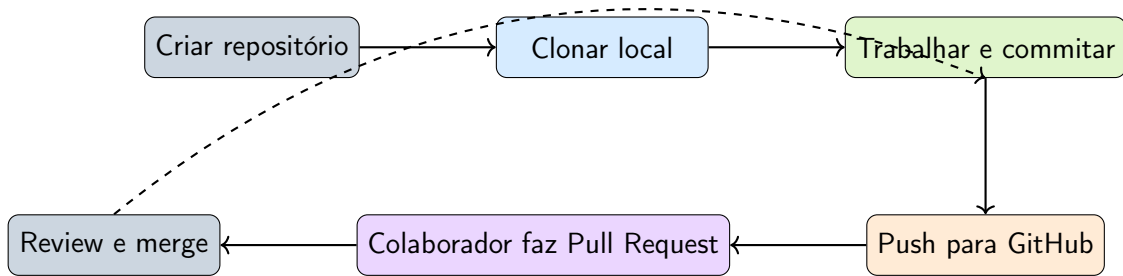
## Boas práticas para cientistas

- Um repositório por projeto/artigo
- README.md com instruções claras
- requirements.txt ou environment.yml
- Dados no repositório ou link para Zenodo/Figshare
- DOI para o repositório via Zenodo

## Importante!

Periódicos como *Nature Methods* e *PLOS Computational Biology* exigem ou recomendam fortemente código disponível publicamente no GitHub.

# GitHub na prática: fluxo de um projeto científico



## Para publicação científica

Ao submeter o artigo: crie uma **Release** no GitHub e registre no Zenodo para obter um DOI permanente e citável.

# O problema da reprodutibilidade

## “Works on my machine” — o pesadelo da ciência computacional

Você publica o código, o revisor tenta rodar, e nada funciona. Versão diferente do Python, biblioteca atualizada que quebrou a API, dependência que não existe mais.

## A solução: empacotar tudo junto

Docker permite criar uma **imagem** que contém:

- O sistema operacional base (Ubuntu, Alpine)
- Python/R com versões exatas
- Todos os pacotes e dependências
- O código e os dados (ou links para eles)
- As configurações do ambiente

## Importante!

Se funciona no seu container, funciona em **qualquer** computador que tenha Docker instalado.



# Docker: conceitos fundamentais

## Definição: Imagem Docker

“Fotografia” de um ambiente completo. Imutável, compartilhável, versionável. Análoga ao snapshot de uma máquina virtual, mas muito mais leve.

## Definição: Container

Uma instância em execução de uma imagem. Isolado do sistema hospedeiro, mas compartilha o kernel do sistema operacional.

## Analogia biológica

**Imagem** = receita de um meio de cultura

**Container** = placa com o meio preparado

Você pode preparar mil placas idênticas a partir da mesma receita, em qualquer laboratório do mundo.

|                 | Máquina Virtual | Docker        |
|-----------------|-----------------|---------------|
| Tamanho típico  | 5–50 GB         | 100 MB – 2 GB |
| Tempo de start  | Minutos         | Segundos      |
| Overhead de CPU | Alto            | Mínimo        |
| Isolamento      | Total           | Parcial       |

# Dockerfile: descrevendo seu ambiente

```
1 # Imagem base com Python 3.12
2 FROM python:3.12-slim
3
4 # Metadados
5 LABEL description="Análise de RNA-seq - Artigo XYZ 2025"
6
7 # Instalar dependências do sistema
8 RUN apt-get update && apt-get install -y \
9     build-essential curl && rm -rf /var/lib/apt/lists/*
10
11 # Copiar e instalar dependências Python
12 COPY requirements.txt .
13 RUN pip install --no-cache-dir -r requirements.txt
14
15 # Copiar código
16 COPY scripts/ /app/scripts/
17
18 # Diretório de trabalho
19 WORKDIR /app
20
21 # Comando padrão
22 CMD ["python", "scripts/analise_principal.py"]
```

Listing 10: Exemplo de Dockerfile para análise bioinformática

# Usando Docker na prática

```
1 # Construir imagem
2 docker build -t minha-analise .
3
4 # Rodar container
5 docker run minha-analise
6
7 # Rodar com pasta local montada
8 docker run -v $(pwd)/dados:/app/dados \
9     minha-analise
10
11 # Ver containers ativos
12 docker ps
13
14 # Parar container
15 docker stop CONTAINER_ID
16
17 # Baixar imagem do Docker Hub
18 docker pull jupyter/scipy-notebook
```

Listing 11: Comandos Docker essenciais

## Docker Hub: imagens prontas

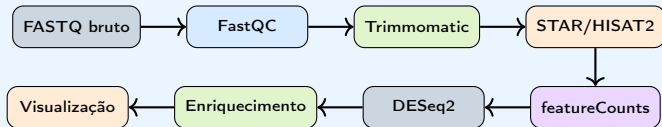
- jupyter/scipy-notebook — Jupyter completo
- rocker/tidyverse — R + tidyverse
- biocontainers/bwa — BWA aligner
- broadinstitute/gatk — GATK oficial
- nfcore/rnaseq — pipeline completo

## Para publicação

Publique sua imagem no Docker Hub e inclua o DOI no artigo para garantia de reprodutibilidade permanente.

# O que é um pipeline bioinformático?

## Pipeline típico de RNA-seq



## Importante!

Rodar cada etapa manualmente é lento, propenso a erros e impossível de reproduzir. Gerenciadores de workflow automatizam e documentam cada passo.

# Snakemake: workflows em Python

## Definição: Snakemake

Gerenciador de workflows baseado em Python. Define **regras** que descrevem como gerar arquivos de saída a partir de entradas, e automaticamente determina a ordem de execução.

```
1 # Snakefile
2 rule alinhamento:
3     input:
4         reads = "dados/{amostra}.fastq.gz",
5         genoma = "genoma/GRCh38.fa"
6     output:
7         bam = "resultados/{amostra}.bam"
8     threads: 8
9     shell:
10         "STAR --runThreadN {threads} "
11         "--genomeDir genoma/ "
12         "--readFilesIn {input.reads} "
13         "--outSAMtype BAM SortedByCoordinate "
14         "--outFileNamePrefix resultados/{wildcards.amostra}"
```

Listing 12: Snakefile simplificado

# Nextflow e nf-core: pipelines prontos

## Nextflow

- Linguagem própria (Groovy-based)
- Executa em HPC, cloud, Docker, Singularity
- Paralelismo automático e escalável
- Muito usado em genômica de larga escala

## nf-core: pipelines prontos

Coleção de **pipelines validados** e mantidos pela comunidade:

- nf-core/rnaseq — RNA-seq completo
- nf-core/chipseq — ChIP-seq
- nf-core/sarek — variantes somáticas
- nf-core/scrnaseq — single-cell

```
nextflow run nf-core/rnaseq \
  -profile docker \
  --input samplesheet.csv \
  --genome GRCh38
```

# LLMs: o novo assistente do cientista

## O que mudou nos últimos anos

Modelos de linguagem (ChatGPT, Claude, Gemini) são capazes de escrever, explicar, debugar e adaptar código científico com alta qualidade.

## O que você pode pedir:

- “Explique o que este código faz”
- “Converta esse script R para Python”
- “Por que estou recebendo este erro?”
- “Escreva um script para analisar este CSV”
- “Otimize esta função para dados grandes”

## Ferramentas recomendadas

- **Claude Code** (Anthropic) — no terminal
- **GitHub Copilot** — no editor de código
- **ChatGPT / Claude.ai** — interface web
- **Gemini no Colab** — integrado ao notebook

# Como usar IA de forma eficaz e responsável

## Boas práticas

- **Contexto é tudo:** descreva seus dados, o formato dos arquivos e o objetivo
- **Itere:** peça ajustes em vez de começar do zero
- **Valide sempre:** teste o código antes de usar em análise real
- **Entenda o que gerou:** não publique código que não compreende

## Cuidados importantes

- IA pode gerar código **plausível mas errado**
- Não envie dados sensíveis (pacientes, sequências proprietárias) para IAs externas
- Verifique estatísticas — modelos erram em detalhes sutis
- Cite ferramentas de IA usadas, quando exigido pelo periódico



# A crise de reprodutibilidade na biologia

## Dado preocupante

Um estudo de 2016 na *Nature* mostrou que mais de **70% dos pesquisadores** não conseguiram reproduzir experimentos de outros laboratórios, e **50%** não conseguiram reproduzir os próprios resultados.

## Causas comuns em estudos computacionais

- Código não disponível ou incompleto
- Versões de software não documentadas
- Dados pré-processados sem descrever as etapas
- Parâmetros ajustados “a mão” sem registro
- Resultados que dependem de semente aleatória não fixada

# O stack da reprodutibilidade

Publicação + DOI (Zenodo, Figshare)

Repositório Git (GitHub) — código versionado

Docker — ambiente fixo e portátil

Conda / venv — dependências documentadas

Jupyter Notebook — análise documentada passo a passo

## Importante!

Reprodutibilidade não é perfeccionismo — é **ciência funcionando**. Qualquer pessoa deve conseguir chegar ao mesmo resultado com os mesmos dados.

# Checklist para um projeto computacional reproduzível

## Estrutura do projeto

```
projeto/  
  README.md           # instrucoes  
  environment.yml      # conda env  
  Dockerfile          # container  
  dados/  
    brutos/           # originais  
    processados/      # derivados  
  scripts/  
    01_preprocessar.py  
    02_analisar.py  
    03_visualizar.py  
  resultados/  
  notebooks/
```

## Ao finalizar o projeto

- ☐ README.md com instruções de execução
- ☐ Ambiente exportado (environment.yml)
- ☐ Imagem Docker publicada
- ☐ Repositório GitHub público
- ☐ Dados em repositório aberto (Zenodo, OSF)
- ☐ DOI gerado para o código
- ☐ Sementes aleatórias fixadas no código

# Roteiro de aprendizagem para biólogos

## Mês 1: Fundamentos

1. Google Colab + Python básico
2. pandas para tabelas de dados
3. matplotlib para gráficos
4. Comandos básicos do terminal

## Mês 2–3: Reprodutibilidade

1. Git e GitHub (repositório próprio)
2. Conda: criar e exportar ambientes
3. Jupyter Lab local

## Mês 4–6: Avançar

1. Docker: criar seu primeiro container
2. Snakemake para automatizar análises
3. Aplicar ao seu problema de pesquisa

## Recursos gratuitos

- **Software Carpentry** — [carpentries.org](https://carpentries.org)
- **Rosalind** — [rosalind.info](https://rosalind.info) (bioinformática)
- **Bioconductor workflows** — [bioconductor.org](https://bioconductor.org)
- **The Turing Way** — livro de ciência reprodutível

# Resumo: o ecossistema computacional do biólogo moderno

